



SAP Application Development (S/4 HANA)

Exception Handling

Content

This chapter is about Exception Handling. You will learn to define a class-based exception and how to raise and handle exceptions.

Prerequisites

You should be familiar with ABAP and principles of ABAP Objects and the navigation in the SAP system and Eclipse. Furthermore, you should have completed the *Introduction to SAP Programming* chapter.

Motivation

Exception Handling is an important technique in advanced ABAP programming.

Exceptions are necessary to make programs more robust, flexible and easy to use.

Lecture notes

Only students who finished module ABAP Objects are able to go through this chapter. This chapter builds upon the work from module ABAP Objects. Students can go on with their account from previous exercises.

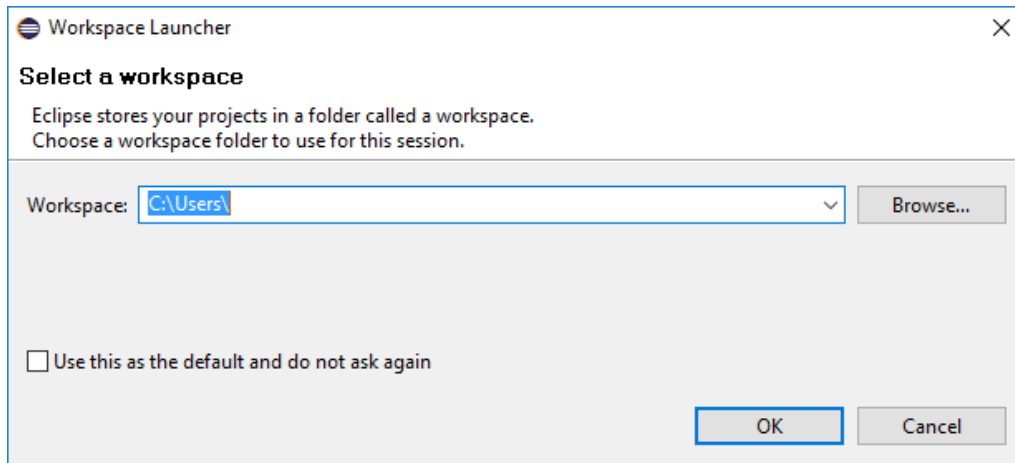
- **Product:** All
- **Level:** Advanced
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Login into the SAP system within Eclipse

Short description: Use Eclipse to login into the SAP system with your username and password

Start Eclipse and select a local workspace where your ABAP project will be stored.

Login



If you have already passed another learning module from this course, navigate to your project and login to the development system using your account from the previous chapters.

Elsewise create a new ABAP Project and Package according to the instructions in the chapters *Introduction to SAP Programming* and *Development of a first program*.

Task 2: Create a new exception Class

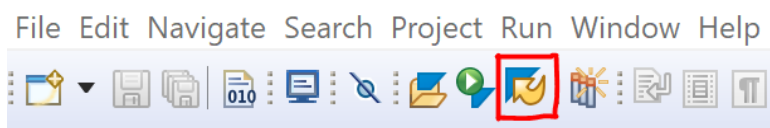
Short description: Use the ABAP Workbench to create class-based exceptions

In this chapter we will implement some exceptions for the configuration of flight data.

- The wrong type of airplane was selected.
- The maximum number of seats is reached for this type of airplane.
- A system exception occurred: reference to an object is initial.

For cases 1 and 2 the necessary classes have to be created. Since both exceptions refer to one airplane type, it is advisable to create a superclass and use inheritance.

Please start the SAP GUI from Eclipse by clicking the SAP GUI Icon in the Menu Bar:



Access to the Class Builder

Choose your project and click OK. Navigate to the Class Builder using transaction code SE24.

Create a new Class for the exception-superclass. The name is **ZCX_000_PLANETYPE**.

Create exceptions

Class Builder: Initial Screen

Class Browser

Object type: ZCX_000_PLANETYPE

Display Change Create

Select **Exception Class** as the Class Type. The Superclass should be set to **CX_STATIC_CHECK** and of type **Public**. Make sure not to mark the class as final and uncheck the "With Message Class" option of the exception class.

Create Class ZCX_PLANETYPE

Class: ZCX_000_PLANETYPE

Superclass: CX_STATIC_CHECK

Description: ZCX_000_PLANETYPE

Inst.Generation: Public

Class Type

☐ Usual ABAP Class

☒ Exception Class

☐ With Message Class

☐ Persistent class

☐ Test Class (ABAP Unit)

☐ Final

Save

Save the class and add it to your package and transport as described in the exercise *Development of a first program*.

Class/Interface: ZCX_000_PLANETYPE Implemented / Inactive

Properties Interfaces Friends Attributes **Texts** Methods Events Types Aliases

Long Text

Exception ID	Text
CX_ROOT	An exception occurred
ZCX_000_PLANETYPE	An exception occurred for the selected planetype.

Exception Text

Change to the tab Texts. Type in a text for the newly created exception ID as shown in the screenshot.

Save your changes.

As the next step, please create an exception for the wrong plane type exception. Use the name **ZCX_\$\$_###_WRONG_PLANETYPE** and use the exception class created above as the Superclass. Mark the class as final.

**Wrong
planetype -
Exception**

Create Class ZCX_000_WRONG_PLANETYPE

Class	ZCX_000_WRONG_PLANETYPE
Superclass	ZCX_000_PLANETYPE
Description	ZCX_000_WRONG_PLANETYPE
Inst.Generation	2 Public

Class Type

☐ Usual ABAP Class

☒ Exception Class

☐ With messages of message classes as exception texts

☐ Persistent class

☐ Test Class (ABAP Unit)

☒ Final

Save

After saving the class change to the tab Attributes. Since we are interested in the planetype that caused the exception, add an attribute named **planetype** of type **string** to the exception attributes.

Important:

Save and activate after you have added the attribute PLANETYPE. You don't have to add the attribute of ZCX_XXX_WRONG_PLANETYPE! This is done automatically after saving and activating.

Class/Interface		ZCX_000_WRONG_PLANETYPE			Implemented / Active			
Properties	Interfaces	Friends	Attributes	Texts	Methods	Events	Types	Aliases
								
Attribute	Level	Visibility	R...	Typing	Associated Ty...			
CX_ROOT	Constant	Public	☐	Type	SOTR_CONC			
TEXTID	Instance A	Public	☒	Type	SOTR_CONC			
PREVIOUS	Instance A	Public	☒	Type Ref T	CX_ROOT			
KERNEL_ERRID	Instance A	Public	☒	Type	S380ERRID			
IS_RESUMABLE	Instance A	Public	☒	Type	ABAP_BOOL			
ZCX_000_PLANETYPE	Constant	Public	☐	Type	SOTR_CONC			
ZCX_000_WRONG_PLANETYPE	Constant	Public	☐	Type	SOTR_CONC			
PLANETYPE	Instance A	Public	☐	Type	STRING			

Important: After adding the attributes you have to save your exception class. Otherwise the attribute cannot be used in an exception text.

For the exception Text you are now able to reference the created attribute **planetype**. Change to the tab Texts. Type in the text

Plane type &planetype& does not exist.

At runtime, the placeholder is used to display the plane type that caused the exception to occur.

Class/Interface	ZCX_000_WRONG_PLANETYPE	Implemented / Active
Properties	Interfaces	Friends
Attributes	Texts	Methods
Events	Types	Alia

Exception ID	Text
CX_ROOT	An exception occurred
ZCX_000_PLANETYPE	An exception occurred for the selected planetype.
ZCX_000_WRONG_PLANETYPE	Plane type &planetype& does not exist.

Repeat these steps for the other exception. This exception is called **ZCX_\$\$_###_MAX_SEATS**.

Create Class ZCX_MAX_SEATS

Class

ZCX_000_MAX_SEATS

Superclass

ZCX_000_PLANETYPE

Description

ZCX_000_MAX_SEATS

Inst.Generation

Public

Class Type

☐ Usual ABAP Class
 ☒ Exception Class
 ☐ With Message Class
 ☐ Persistent class
 ☐ Test Class (ABAP Unit)

☒ Final

Save

**Maximum
number of
seats -
Exception**

It has an additional public instance attribute called **max_seats** of **Type I** (Integer) to store the maximum number of seats which are available in the airplane. The exception text should be

Maximum number of available seats &max_seats& is reached. No more bookings possible.

Save and activate your changes.

Task 3: Create an ABAP report and learn how to raise and catch exceptions.

Short description: Create an ABAP report to raise the previously created exceptions.

If you have already done the exercise **for Module ABAP Objects**, you can copy your class 'Z_\$\$_###_CLASS_INCLUDE' to 'Z_\$\$_###_CLASS_EXCEPTIONS_INCLUDE' and modify the code for this exercise.

Create a new ABAP program 'Z_\$\$_###_EXCEPTIONS'.

You will create a new local class with the purpose to represent airlines.

Create a new include program Z_\$\$_###_CLASS_EXCEPTIONS_INCLUDE.

The class should have the four private instance attributes `gv_name` (representing the flight name, e.g. Munich or Hamburg), `gv_planetype` (representing the planetype, e.g. Boeing 737 or Airbus 380), `gv_max_seats`, `gv_occ_seats`. For simplicity, set the default value of the maximum number of seats to 300 and the number of already occupied seats to 255.

Additionally, implement the following public instance methods: `set_attributes` with importing parameters `im_name` and `im_planetype`, `get_attributes`, `check_planetype` raising exception 'ZCX_\$\$_###_WRONG_PLANETYPE' and method `add_seats` raising exception 'ZCX_\$\$_###_MAX_SEATS'. As both exceptions are derived from exception 'ZCX_\$\$_###_PLANETYPE', it is sufficient to specify this exception in the raising clause.

**Defining the
used
exceptions**

```
CLASS lcl_airplane DEFINITION.

PUBLIC SECTION.
METHODS: set_attributes
          IMPORTING
            im_name TYPE string
            im_planetype TYPE string,
          get_attributes,
          check_planetype
            RAISING zcx_000_planetype,
          add_seats
            IMPORTING im_add TYPE i
            RAISING zcx_000_planetype.

PRIVATE SECTION.
DATA: gv_name TYPE string,
      gv_planetype TYPE string,
      gv_max_seats TYPE i VALUE 300,
      gv_occ_seats TYPE i VALUE 255.

ENDCLASS.
```

```
CLASS lcl_airplane IMPLEMENTATION.
METHOD set_attributes.
  gv_name = im_name.
  gv_planetype = im_planetype.
ENDMETHOD.
```

The implementation of method `set_attributes` should set global attributes `gv_name` and `gv_planetype`.

Method `get_attributes` should write these attributes plus – if it is below the number of available seats - the number of occupied seats `gv_occ_seats` to the output screen. Method `check_planetype` has to check whether the entered plane type is in the list of valid plane types (here 'Boeing 737' and 'Airbus 380'). Finally, method `add_seats` has to increase the number of occupied seats by the entered number. If the total number of occupied seats exceeds the maximum available number of seats, exception `ZCX_$$_###_MAX_SEATS` should be raised.

```

CLASS lcl_airplane IMPLEMENTATION.
METHOD set_attributes.
  gv_name = im_name.
  gv_planetype = im_planetype.
ENDMETHOD.

METHOD get_attributes.
WRITE: / 'Flight name: ', gv_name,
        'Flight type: ', gv_planetype.
"display no. of occupied seats, only if it is below the maximum no. of seats
IF gv_occ_seats <= gv_max_seats.
  WRITE: / 'Number of occupied seats: ', gv_occ_seats.
ENDIF.
ENDMETHOD.

METHOD check_planetype.
TRANSLATE gv_planetype TO UPPER CASE.
"Check if the planetype is in the list of valid planetypes
IF gv_planetype <> 'BOEING 737' AND gv_planetype <> 'AIRBUS 380'.
  RAISE EXCEPTION TYPE zcx_000_wrong_planetype
    EXPORTING planetype = gv_planetype.
ENDIF.
ENDMETHOD.

METHOD add_seats.
gv_occ_seats = gv_occ_seats + im_add.
IF gv_occ_seats > gv_max_seats.
  RAISE EXCEPTION TYPE zcx_000_max_seats
    EXPORTING max_seats = gv_max_seats.
ENDIF.
ENDMETHOD.
ENDCLASS.

```

Change back to your ABAP program 'Z_\$\$_###_EXCEPTIONS' and call the just created include typing

```
INCLUDE z_000_class_exceptions_include.
```

We want to enable the dynamic input of parameters for our report. Therefore, we define the following input parameters:

```

PARAMETERS:
fl_name TYPE string,      "flight name
pl_type TYPE string,      "plane type
add_seat TYPE i.          "number of seats to be added

```

*Advice: To change the selection texts (these are the field labels, which will be displayed on the output screen) of parameters in Eclipse, you have to do the following: **Right-click on the parameter → Open Others → Text elements***

Additionally, the following data declarations are needed:

```

DATA: r_plane TYPE REF TO lcl_airplane,
      it_plane_list TYPE TABLE OF REF TO lcl_airplane,
      lo_excp TYPE REF TO zcx_000_planetype,
      lv_excp_text TYPE string.

```

Now start the block 'START-OF-SELECTION'. Create an object with command **CREATE OBJECT** that is referenced by variable **r_plane**. Call method **set_attribute**

using this reference and assign name and plane type to the new object using the corresponding parameters of the method.

Advice: Use Shift + Enter for code completion

```
START-OF-SELECTION.

CREATE OBJECT r_plane.
r_plane->set_attributes(
  EXPORTING
    im_name      = fl_name
    im_planetype = pl_type ).
```

Now we want to find out, whether the entered plane type is in our list of valid plane types 'Boeing 737' and 'Airbus 380' by calling method check_planetype. As we have implemented the exception ZCX_\$\$\$_WRONG_PLANETYPE, we now have to catch the corresponding exception object.

Additionally, exception handling has to be implemented.

For this, you first have to check, whether the exception object was created. In case that the exception object is not initial, get the exception text and display it on the output screen.

```
TRY.
  r_plane->check_planetype( ).
  CATCH zcx_000_planetype INTO lo_excp.

  IF lo_excp IS NOT INITIAL.
    lv_excp_text = lo_excp->get_text( ).
    WRITE: lv_excp_text, /.
    RETURN.
  ENDIF.
ENDTRY.
```

In a next step, we want to increase the number of occupied seats in the airplane by the number seat which was entered on the input screen. For this, you have to call method add_seat and implement an error handling in the same way as for method check_planetype.

```
TRY.
  r_plane->add_seats( im_add = add_seat ).
  CATCH zcx_000_planetype INTO lo_excp.

  IF lo_excp IS NOT INITIAL.
    lv_excp_text = lo_excp->get_text( ).
    WRITE: lv_excp_text, /.
  ENDIF.
ENDTRY.
```

Finally, the attributes of the airplane should be displayed.

```
"Everything went right, display attributes of airplane
r_plane->get_attributes( ).
```

Finally, execute  your report after saving and activating your changes.

What happens, if you exceed the maximum number of seats? What happens if you enter a plane type which is not in the list of valid airplanes (Airbus 380, Boeing 737)?

Can you enter any random flight name?

Task 4

Short description: Raise and catch a system exception.

An error, that occurs frequently in ABAP Object programming is that the reference to an object is initial. The exception which is raised in this case is the system exception CX_SY_REF_IS_INITIAL.

Try to call method `set_attributes` of class `lcl_airplane` in your report 'Z_\$\$_###_EXCEPTIONS' without having created an object `r_plane` as in the tasks before.

```
REPORT z_000_exceptions.

INCLUDE z_000_class_exceptions_include.

PARAMETERS:
fl_name TYPE string, "flight name
pl_type TYPE string,  "plane type
add_seat TYPE i.      "number of seats to be added

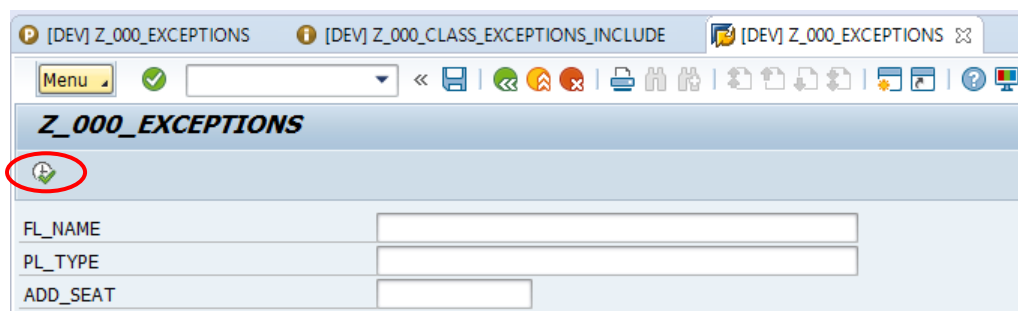
DATA: r_plane TYPE REF TO lcl_airplane,
      it_plane_list TYPE TABLE OF REF TO lcl_airplane,
      lo_excp TYPE REF TO zcx_000_planetype,
      lv_excp_text TYPE string.

START-OF-SELECTION.

***CREATE OBJECT r_plane

r_plane->set_attributes(
  EXPORTING
    im_name      = fl_name
    im_planetype = pl_type ).
```

Save, activate and run the program.



The screenshot shows the SAP ABAP development environment. At the top, there are three tabs: '[DEV] Z_000_EXCEPTIONS', '[DEV] Z_000_CLASS_EXCEPTIONS_INCLUDE', and '[DEV] Z_000_EXCEPTIONS'. Below the tabs is a toolbar with various icons. The main area displays the report name 'Z_000_EXCEPTIONS'. Below the report name, there is a green play icon (execute button) which is circled in red. Below the execute button, there are three input fields: 'FL_NAME', 'PL_TYPE', and 'ADD_SEAT'.

By clicking on the execute button, the following exception will occur:

Runtime Error - Description of Exception

Long Text Debugger

Category	ABAP Programming Error
Runtime Errors	OBJECTS_OBJREF_NOT_ASSIGNED
Except.	CX_SY_REF_IS_INITIAL
ABAP Program	Z_000_EXCEPTIONS
Application Component	Not assigned
Date and Time	24.08.2016 16:54:42

Short Text

Access using a 'ZERO' object reference is not possible.

What happened?

Error in the ABAP Application Program

The current ABAP program "Z_000_EXCEPTIONS" had to be terminated because it has come across a statement that unfortunately cannot be executed.

Error analysis

An exception has occurred which is explained in more detail below. The exception, which is assigned to class 'CX_SY_REF_IS_INITIAL' was not caught and therefore caused a runtime error. The reason for the exception is:
 You are trying to access a component with a 'ZERO' object reference (points to 'nothing'). Variable: "R_PLANE".
 An object reference must point to an object (an instance of a class) before it can be used to access a component. The reference has either never been set, or it was set to 'ZERO' with a CLEAR statement.

Now, catch the exception which is described in the runtime error above into an exception object.

Like every exception class, CX_SY_REF_IS_INITIAL is derived from a superclass, in this case it is CX_DYNAMIC_CHECK.

Enclose the exception within a try – catch – entry structure and write the longtext of the exception to the output screen and return.

```

REPORT z_000_exceptions.

INCLUDE z_000_class_exceptions_include.

PARAMETERS:
fl_name TYPE string, "flight name
pl_type TYPE string,  "plane type
add_seat TYPE i.      "number of seats to be added

DATA: r_plane TYPE REF TO lcl_airplane,
      it_plane_list TYPE TABLE OF REF TO lcl_airplane,
      lo_excp TYPE REF TO zcx_000_planetype,
      lv_excp_text TYPE string.

START-OF-SELECTION.

DATA lo_sy_excp TYPE REF TO cx_dynamic_check.
TRY.
  r_plane->set_attributes(
    EXPORTING
      im_name      = fl_name
      im_planetype = pl_type ).
  CATCH cx_sy_ref_is_initial INTO lo_sy_excp.
    IF lo_sy_excp IS NOT INITIAL.
      lv_excp_text = lo_sy_excp->get_longtext( ).
      WRITE: lv_excp_text, /.
    RETURN.
  ENDIF.
ENDTRY.

```

Save, activate and run your program and click on the execute button.

The screenshot shows the SAP ABAP development environment. At the top, there are three tabs: [DEV] Z_000_EXCEPTIONS, [DEV] Z_000_CLASS_EXCEPTIONS_INCLUDE, and [DEV] Z_000_EXCEPTIONS. Below the tabs is a menu bar with a 'Menu' button and a dropdown arrow. To the right of the menu bar is a toolbar with various icons. The main area displays the title 'Z_000_EXCEPTIONS' in a blue header. Below the header, there is a green circle with a white play icon, which is circled in red. Below this icon are three input fields: 'FL_NAME', 'PL_TYPE', and 'ADD_SEAT'.

The output should be the following:

Z_000_EXCEPTIONS

Z_000_EXCEPTIONS 1

An attempt was made to execute a dynamic method call on an initial(NULL-) object reference. The reference must refer to an object.

Challenge:

Short description: Extend your report and class with a check for the flight name. The flight name will be accepted if it is 'Hamburg' or 'Munich'.